

Agent Design Rationale

I designed the four agents around a decisive separation of tasks. Instead of giving each agent a generic and interchangeable persona, I tried to give a backstory to each one that would shape their execution strategy. The Query Expander had a backstory that told it to never map a single research question into a single query. It also instructs it to never leave a question undecomposed. The source hunter had a backstory that tells it that the first search result isn't usually the best one. It tells it that it should be suspicious of stopping early and to dig deeper. The synthesizer is framed as an analyst whose job is to compare and contrast across different papers instead of just summarizing them one at a time. The reporter writer is shaped to be an editor who believes its citations need to be of a higher quality. This design choice for the agents was not sufficient on its own, however. The source hunter initially defaulted to a shallow retrieval strategy. It started to focus on a single paper despite a backstory that told it not to do so. To fix this, the corresponding task description was tightened with concrete requirements to check several papers. The lesson here was that the instinct of an agent based on its backstory is not sufficient and needs to be operationalized into something that can be checked in the task description.

ReAct-Style Reasoning in the Agent Output

The source hunter is the only agent in this crew that calls a tool. Its execution trace is where the ReAct pattern is repeated multiple times and is the most visible. Below is an excerpt from my terminal output during a run of main.py:

```
| Agent: Source Hunter

| Task: Using the search_papers tool, search the corpus for every sub-question and keyword variant produced in the previous task. You MUST issue at least 6 separate search_papers calls with different queries -- do not stop after 1-2 searches even if the first results look relevant. Use k=8 on each call so you have a wide pool of candidates to choose from.

| After each search, track which paper(s) the results came from. Before finishing, check your running list of passages: if 4 or more of them come from the same single paper, you have NOT searched broadly enough -- run additional searches using different keywords or phrasing aimed specifically at other papers in the corpus. The corpus includes Wooldridge, Wang/Xi surveys, ReAct, Chain-of-Thought, Tree of Thoughts, Reflexion, CAMEL, Generative Agents, AutoGen, Toolformer, RAG (2020 and 2023 survey), Planning Abilities, and Constitutional AI -- if any of these are clearly relevant to the question and you haven't found them yet, search for them by name.

| Stop only once you have 8-12 non-redundant passages drawn from at least 6 different

🔧 Tool Execution Started (#1)

| Tool: search_research_papers

| Args: {'query': 'AI agent theory', 'k': 8}

🔧 Tool Execution Started (#6)

| Tool: search_research_papers

| Args: {'query': 'collaborative AI agents', 'k': 8}

✅ Tool Execution Completed (#6)

| Tool Completed
```

| Tool: search_research_papers

| Output: Found 8 relevant passages:

This excerpt is from the Source hunter agent which is executing the search_papers tool. One note to make on format is that this version of CrewAI's verbose output renders each step as a structured panel rather than printing the steps like "Thought:". So the agent's internal reasoning between the tool calls is not directly visible. That behavior is the evidence that instead of stopping after one or two search, which it did before, the agent ran at least six distinct and differently worded queries in sequence. This is proof that the task instruction to keep searching worked instead of settling for early results. Call #6 returned a healthy spread across five different papers, while the next call returned eight passages entirely from a single paper.

What worked, What didn't, and What I would change

What worked was that the four-stage pipeline ran end-to-end without errors, and CrewAI's task level context parameter correctly passed each agent's output forward. By the time the reporter ran, it had visibility into the original question. Reports were consistently organized by themes rather than by paper as this was a specific design goal reflected in both the task descriptions and the report writer's backstory. What didn't work was that the citation accuracy turned out to be a much harder problem than I expected. Across several regenerations of the same research question, the report writer repeatedly fabricated plausible-looking citations involving two survey papers with similar titles. After I instructed it to cite using the exact paper_id it returned by the search tool instead of investing other author names. These errors were concentrated specifically on the two papers whose topics and titles were most similar. What I would change is to instruct the model to more precisely handle the citations and not to use ones outside of the model. I managed to get a clean run but I'm currently unsure if it will default to using other articles again.

Production Considerations

Taking this assignment to a full production system would require at least three changes. One change is to have automated citation verification. This is the most direct lesson that I have learned from this assignment in that there needs to be a deterministic code-based check after the report writer finishes, which parses every cited paper_id out the generated report and cross references it with the actual set of paper_ids. Environment and dependency reproducibility was another issue I ran into. A production deployment would need to be a locked and tested python version and have its dependency set from day one. Credential handling discipline would also be helpful as working through an environment configuration can end up exposing an API key. I would need a production system that needs to guarantee that keys are never logged are echoed anywhere and pair it with an automated rotation for accidental exposure.

